

普及组模拟 23

一、单项选择题 (共 15 题, 每题 2 分, 共计 30 分; 每题有且仅有一个正确选项)

1. 下列有关计算机说法错误的是 ()。

- A. 中央处理器简称为 *CPU*, 它是计算机系统的核心
- B. 存储器可分为内存储器 and 外存储器
- C. 键盘、鼠标、摄像头为输入设备
- D. 显示器、打印机、扫描仪为输出设备

2. 1GB 代表的字节数量是 ()。

- A. 2 的 10 次方
- B. 2 的 20 次方
- C. 2 的 30 次方
- D. 2 的 40 次方

3. 归并排序的时间复杂度是 ()。

- A. $O(n)$
- B. $O(2n)$
- C. $O(n\log n)$
- D. $O(n^2)$

4. 下列与二进制数 11.11 相等的八进制数为 ()。

- A. 3.75
- B. 3.6
- C. 3.7
- D. 3.65

5. 阅读下面程序

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d",&n);
    int k=0,ans=0;
    while(k<=n){
        ans+=k;
        k+=2;
    }
    printf("%d",ans);
    return 0;
}
```

如果输入 12, 则输出为 ()。

- A. 12
- B. 0
- C. 42
- D. 72

6. 若 $x = \text{true}$, $y = \text{true}$, $z = \text{false}$, 则下面的式子的值为 *false* 的是 ()。

- A. $(x \& \& y) \& \& (!z)$
- B. $(!z) \& \& x || y$
- C. $!(x || y) || z$
- D. $x \& \& (y || z)$

7. 对于以下递归函数 *fun*, 调用 *fun*(5), 其返回值为 ()。

```
int fun(int x){
    if(x<=0) return 0;
    return fun(x-1)+x/2;
}
```

A. 5

B. 6

C. 6.5

D. 7.5

8. 下列关于树的说法错误的是()。

A. 树的叶子节点的度为0

B. 二叉树是每个结点最多有两个子树的树结构

C. 高度为 h ,并且由 2^h 个结点组成的二叉树,称为满二叉树

D. 树是一种数据结构,它是由 $n(n \geq 1)$ 个有限结点组成一个具有层次关系的集合

9. 二进制数00101100与二进制10011100的和为()。

A. 11000000

B. 10101000

C. 10110000

D. 11001000

10. 已知小明要做三件事情,其中 A 需要30分钟, B 需要20分钟, C 需要15分钟,其中 C 必须在完成 A 后开始, A 和 B 不能同时进行,小明最多可以同时做两件事情,问做完这三件事情需要的最小的时间是()

A. 30

B. 45

C. 50

D. 65

11. 如果一个字符串 $S = "ABBC"$,则它的所有子串一共有()种

A. 8

B. 9

C. 10

D. 11

12. 判断一个大小为 N 的数是否为质数,最小的时间复杂度为()。

A. $O(N)$

B. $O(1)$

C. $O(N \log N)$

D. $O(\sqrt{N})$

13. 5136和6144的最大公约数是()。

A. 2

B. 12

C. 48

D. 64

14. 已知一颗二叉树有50个结点,则下列数字中,不可能为这颗树的高度(记根所在的那一层高度为1)的是()

A. 5

B. 7

C. 10

D. 50

15. 五个人站成一排,如果甲和乙不在两端,则共有()种站法

A. 24种

B. 36种

C. 42种

D. 72种

二、阅读程序(程序输入不超过数组或字符串定义的范围;判断题正确填 $\sqrt{}$,错误填 $\times$;除特殊说明外,判断题1.5分,选择题3分,共计40分)

1. 阅读以下程序:

```
1 #include <iostream>
2 using namespace std;
3 int a[100050], b[100050], c[1000000];
4 int n;
5 int main()
6 {
7     int n;
```

```

8    cin >> n;
9    int ans = 0;
10   for (int i = 1; i <= n; i++)
11   {
12       cin >> a[i];
13   }
14   for (int i = 1; i <= n; i++)
15   {
16       cin >> b[i];
17   }
18   for (int i = 1; i <= n; i++)
19   {
20       c[a[i]]++;
21   }
22   for (int i = 1; i <= n; i++)
23   {
24       if (c[b[i]])
25       {
26           ans++;
27           c[b[i]] = 0;
28       }
29   }
30   cout << ans;
31   return 0;
32 }

```

保证输入的 $n < 10000$, $a_i, b_i < 1000000$ 完成下面的判断和单选题

●**判断题**

1. 1) 可以将第一个 *for* 循环的内容和第二个 *for* 循环的内容合并在一起以减小代码的篇幅,即删去 13、14、15 行,将第 11 行改为 $cin >> a[i] >> b[i]$ ()
2. 2) 第七行写成 $int\ n = 0$; 不会影响最终答案 ()
3. 3) 如果将第 17 行的 a 改为 b ,第 20 行和第 22 行的 b 改为 a 不会影响最终答案 ()

●**单选题**

4. 1) 若输入的 n 为 5,接下来输入 8, 5, 5, 6, 4, 1, 2, 3, 4, 5 则该程序输出的是 ()
 A. 0 B. 1 C. 2 D. 3
5. 2) 若输入的 n 为 5,最终答案是 3,则该程序除 n 之外的输入可能是
 A. 2 5 6 4 7 3 5 4 6 4
 B. 1 2 3 4 5 4 5 6 7 8
 C. 9 5 2 2 1 5 2 2 1 9
 D. 8 2 1 2 2 2 2 2 9 4
6. 3) 若输入的 n 为 15,接下来输入

1 5 3 2 1 4 5 6 4 5 2 1 2 5 7
2 3 6 5 8 9 6 5 2 5 3 5 4 2 3

则该程序输出的是()

A. 2

B. 3

C. 4

D. 5

2. 阅读以下程序

```
1 #include <iostream>
2 #include <cstring>
3 using namespace std;
4 const int N = 1e3 + 100;
5 char a[N][N];
6 char b[N][N];
7 void Sort(int l, int r)
8 {
9     if (l >= r)
10         return;
11     int mid = (l + r) / 2;
12     Sort(l, mid);
13     Sort(mid + 1, r);
14     int i = l, j = mid + 1;
15     int k = l;
16     while (i <= mid && j <= r)
17     {
18         if (strcmp(a[i], a[j]) <= 0)
19         {
20             strcpy(b[k], a[i]);
21             k++;
22             i++;
23         }
24         else
25         {
26             strcpy(b[k], a[j]);
27             k++;
28             j++;
29         }
30     }
31     while (i <= mid)
32     {
33         strcpy(b[k], a[i]);
34         k++;
35         i++;
```

```

36     }
37     while (j <= r)
38     {
39         strcpy(b[k], a[j]);
40         k++;
41         j++;
42     }
43     for (int ii = l; ii <= r; ii++)
44         strcpy(a[ii], b[ii]);
45 }
46 int main()
47 {
48     int n;
49     cin >> n;
50     for (int i = 1; i <= n; i++)
51         cin >> a[i];
52     Sort(1, n);
53     char c[10000];
54     int len = 0;
55     for (int i = 1; i <= n; i++)
56     {
57         c[len++] = a[i][strlen(a[i]) - 1];
58         if (c[len - 1] >= 'a' && c[len - 1] <= 'z')
59         {
60             c[len - 1] -= 32;
61         }
62     }
63     printf("%s", c);
64 }

```

完成下面的判断题和单选题

●判断题

1. 1) 第 18 行的 `<=` 可以改为 `<` ()
2. 2) 输出中可能存在小写字母 ()
3. 3) 第 60 行的 `c[len - 1]` 可以写成 `c[len --]` ()
4. 4) 本程序中的所有输入的字符串与输出的字符串有可能全部一样 ()

●单选题

5. 1) 如果输入为 5 *add aaa aab aas ayt*, 则输出为 ()

A. <i>AbSdT</i>	B. <i>ADSBT</i>	C. <i>ABSdT</i>	D. <i>AAAYD</i>
-----------------	-----------------	-----------------	-----------------
6. 2) 如果输入为 12 *A b c d e F g H i j K l*, 则输出为 ()

A. <i>ABCFHKDEGIJL</i>	B. <i>AFHKBCDEGIJL</i>
------------------------	------------------------

3. 阅读以下程序

```
1 #include <iostream>
2 using namespace std;
3 const int N = 100005;
4
5 int n, cnt, rt;
6
7 struct node
8 {
9     int ls, rs, val, siz, cnt;
10 } nod[N];
11
12 void Push_up(int x)
13 {
14     int ls = nod[x].ls, rs = nod[x].rs;
15     nod[x].siz = nod[ls].siz + nod[rs].siz + nod[x].cnt;
16 }
17
18 void Insert(int &rt, int x)
19 {
20     if (rt == 0)
21     {
22         rt = ++cnt;
23         nod[rt].siz = nod[rt].cnt = 1;
24         nod[rt].val = x;
25         return;
26     }
27     if (nod[rt].val == x)
28     {
29         nod[rt].cnt++;
30         nod[rt].siz++;
31         return;
32     }
33     if (nod[rt].val < x)
34     {
35         Insert(nod[rt].rs, x);
36         Push_up(rt);
37         return;
38     }
39     if (nod[rt].val > x)
40     {
```

```

41         Insert(nod[rt].ls, x);
42         Push__up(rt);
43         return;
44     }
45 }
46
47 int Findmin(int &rt)
48 {
49     if (nod[rt].ls)
50     {
51         int ret = Findmin(nod[rt].ls);
52         Push__up(rt);
53         return ret;
54     }
55     int ret = rt;
56     rt = nod[rt].rs;
57     return ret;
58 }
59
60 void Delete(int &rt, int x)
61 {
62     if (nod[rt].val == x)
63     {
64         if (nod[rt].cnt > 1)
65         {
66             nod[rt].cnt--;
67             nod[rt].siz--;
68             return;
69         }
70         else
71         {
72             if (nod[rt].ls == 0)
73             {
74                 rt = nod[rt].rs;
75                 return;
76             }
77             if (nod[rt].rs == 0)
78             {
79                 rt = nod[rt].ls;
80                 return;
81             }
82             else
83             {

```

```

84             int nd = Findmin(nod[rt].rs);
85             nod[rt].val = nod[nd].val;
86             nod[rt].cnt = nod[nd].cnt;
87             Push__up(rt);
88             return;
89         }
90     }
91 }
92 if (nod[rt].val < x)
93 {
94     Delete(nod[rt].rs, x);
95     Push__up(rt);
96     return;
97 }
98 if (nod[rt].val > x)
99 {
100     Delete(nod[rt].ls, x);
101     Push__up(rt);
102     return;
103 }
104 }
105
106 int Getk(int rt, int x)
107 {
108     if (nod[rt].val == x)
109         return nod[nod[rt].ls].siz + 1;
110     else if (nod[rt].val < x)
111         return nod[nod[rt].ls].siz + nod[rt].cnt + Getk(nod[rt].rs, x);
112     else if (nod[rt].val > x)
113         return Getk(nod[rt].ls, x);
114 }
115
116 int Getkth(int rt, int x)
117 {
118     if (nod[nod[rt].ls].siz + 1 <= x && nod[nod[rt].ls].siz + nod[rt].cnt >
= x)
119         return nod[rt].val;
120     if (nod[nod[rt].ls].siz + 1 > x)
121         return Getkth(nod[rt].ls, x);
122     if (nod[nod[rt].ls].siz + nod[rt].cnt < x)
123         return Getkth(nod[rt].rs, x - (nod[nod[rt].ls].siz + nod[rt].cnt));
124 }
125

```



```

126 int Getpre(int rt, int x)
127 {
128     int p = rt, ans;
129     while (p)
130     {
131         if (x <= nod[p].val)
132             p = nod[p].ls;
133         else
134             ans = p, p = nod[p].rs;
135     }
136     return nod[ans].val;
137 }
138 int main()
139 {
140     scanf("%d", &n);
141     for (int i = 1; i <= n; i++)
142     {
143         int opt, x;
144         scanf("%d%d", &opt, &x);
145         if (opt == 1)
146             Insert(rot, x);
147         if (opt == 2)
148             Delete(rot, x);
149         if (opt == 3)
150             printf("%d\n", Getk(rot, x));
151         if (opt == 4)
152             printf("%d\n", Getkth(rot, x));
153         if (opt == 5)
154             printf("%d\n", Getpre(rot, x));
155     }
156 }

```

完成下面的判断和单选题

●判断题

1. 1) 该代码实现了一个链式的数据结构 ()
2. 2) 当 $opt = 2$ 时, 程序会删掉排名为 x 的点 ()
3. 3) 函数 $Getk$ 的时间复杂度是 $O(\log N)$ ()

●单选题

4. 1) 若输入为
 - 5
 - 1 2
 - 1 3
 - 1 4

2 3

3 4

该程序会输出 ()

A. 0

B. 1

C. 2

D. 3

5.2) 若输入为

10

1 2

1 3

1 4

1 5

1 6

2 5

3 6

1 1

4 3

3 3

该程序会输出 ()

A. 4 2 4

B. 5 2 4

C. 4 3 3

D. 5 3 4

6.3) 若输入

15

1 8

1 9

1 7

1 7

2 8

1 6

1 10

1 5

3 5

5 10

1 3

4 4

1 11

5 4

4 5

该程序会输出 ()

A. 1 8 7 3 7

B. 1 8 6 3 7

C. 1 9 7 3 7

D. 2 9 7 2 7

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

1

一个人想要从从 n 点到达 k 点 (n 和 k 在同一个坐标轴上, 对应坐标为 n 和 k , 并且 $n, k \leq 100000$), 现在可以进行下列两种操作中的一种:

1. 从 x 移动到 $x - 1$ 或者 $x + 1$
2. 从 x 移动到 $2x$
问到达 k 点需要的最小操作次数是多少
例如输入 $n = 5, k = 17$
应当输出 4
试补全程序。

```
1 #include <iostream>
2 #include <string.h>
3 using namespace std;
4 const int N = 1e5 + 10;
5 int q[N], h[N];
6 int bfs(int a, int b)
7 {
8     memset(h, -1, sizeof h);
9     q[0] = a;
10    h[a] = 0;
11    int hh = 0, tt = 0;
12    while (hh <= tt)
13    {
14        int t = ①;
15        if (②)
16            return h[b];
17        if (t + 1 <= 1e5 && ③)
18        {
19            q[++tt] = t + 1;
20            h[t + 1] = h[t] + 1;
21        }
22        if (t - 1 >= 0 && h[t - 1] == -1)
23        {
24            ④ h[t - 1] = h[t] + 1;
25        }
26        if (t * 2 <= 1e5 && h[2 * t] == -1)
27        {
28            ⑤ h[t * 2] = h[t] + 1;
29        }
30    }
31    return h[b];
32 }
33
34 int main()
35 {
36    int a, b;
```

```

37     cin >> a >> b;
38     cout << bfs(a, b);
39     return 0;
40 }

```

1. 1) ①处应填 ()

A. $q[++hh]$ B. $q[hh++]$ C. $q[hh--]$ D. $q[--hh]$

2. 2) ②处应填 ()

A. $hh == tt$ B. $hh == q[tt]$ C. $t == b$ D. $t == h[b]$

3. 3) ③处应填 ()

A. $h[t - 1] == -1$ B. $h[t - 1] == 0$
C. $h[t + 1] == -1$ D. $h[t + 1] == 0$

4. 4) ④处应填 ()

A. $q[++tt] = t - 1;$ B. $q[tt++] = t - 1;$
C. $q[tt--] = t + 1;$ D. $q[--tt] = t + 1;$

5. 5) ⑤处应填 ()

A. $q[++tt] = t * 2;$ B. $q[tt++] = t * 2;$
C. $q[tt--] = t / 2;$ D. $q[--tt] = t / 2;$

2

现要求出在 $[a, b]$ 中所有的整数中, 每个数码 (0~9) 各出现了多少次。(即求出所有满足 $a \leq i \leq b$ 的整数 i 中, 0 到 9 十个数字各出现了多少次)

例如输入 $a=1, b=2$

应当输出 0 1 1 0 0 0 0 0 0 0

输入 $a=1, b=99$

应当输出 9 20 20 20 20 20 20 20 20 20

试补全程序。

```

1 #include <iostream>
2 using namespace std;
3
4 const int MaxN = 15;
5 int f[MaxN][10][10];
6 int a, b;
7
8 int q_pow(int a, int b)
9 {
10     int res = 1;
11     while (b)
12     {
13         if (b & 1)

```

```

14         res *= a;
15         b >>= 1;
16         ①
17     }
18     return res;
19 }
20
21 void init()
22 {
23     for (int i = 0; i <= 9; i++)
24         f[1][i][i] = 1;
25     for (int i = 2; i <= 13; i++)
26     {
27         for (int j = 0; j <= 9; j++)
28         {
29             for (int k = 0; k <= 9; k++)
30             {
31                 for (int l = 0; l <= 9; l++)
32                 {
33                     f[i][j][l] += f[i - 1][k][l];
34                 }
35             }
36             f[i][j][j] += ②;
37         }
38     }
39 }
40
41 int solve(int x, int num)
42 {
43     int digit[15];
44     int len = 0, ans = 0;
45     memset(digit, 0, sizeof(digit));
46     while (x)
47     {
48         digit[++len] = x % 10;
49         x /= 10;
50     }
51     for (int i = 1; i < len; i++)
52     {
53         for (int j = 1; j <= 9; j++)
54         {
55             ans += f[i][j][num];
56         }

```

```

57     }
58     for (int i = 1; i < ③; i++)
59     {
60         ans += f[len][i][num];
61     }
62     for (int i = len - 1; i >= 1; i--)
63     {
64         for (int j = 0; j < digit[i]; j++)
65         {
66             ans += f[i][j][num];
67         }
68         for (int j = len; j > i; j--)
69         {
70             if (④)
71                 ans += digit[i] * q_pow(10, i - 1);
72         }
73     }
74     return ans;
75 }
76
77 int main()
78 {
79     scanf("%d%d", &a, &b);
80     init();
81     for (int i = 0; i <= 9; i++)
82     {
83         printf("%lld ", ⑤);
84     }
85     return 0;
86 }

```

1. 1) ①处应填 ()

- A. $a *= a;$ B. $a ++;$ C. $a *= b;$ D. $a --;$

2. 2) ②处应填 ()

- A. $q_pow(2, i - 1);$ B. $q_pow(2, i);$
 C. $q_pow(10, i - 1);$ D. $q_pow(10, i);$

3. 3) ③处应填 ()

- A. $digit[len + 1]$ B. $digit[len - 1]$ C. $digit[len]$ D. $digit[len - i]$

4. 4) ④处应填 ()

- A. $digit[i] != num$ B. $digit[i] == num$
 C. $digit[j] != num$ D. $digit[j] == num$

5.5) ⑤处应填()

A. $\text{solve}(b+1, i) - \text{solve}(a, i)$

B. $\text{solve}(b+1, i) - \text{solve}(a-1, i)$

C. $\text{solve}(b, i) - \text{solve}(a-1, i)$

D. $\text{solve}(b, i) - \text{solve}(a, i)$